Secure_File_Transfer_Monitoring_System

NAME: DHEERAJ
DATE: 17-03-2026
PROJECT: Secure_File_Transfer_Monitoring_System
GitHub Link:
https://github.com/Boltx099/Secure_File_Transfer_Monitoring_System

---

1. Introduction

The Secure File Transfer Monitoring System is a Python-based application designed to monitor file system activity in real time and detect suspicious or unauthorized file operations.

It provides visibility into file creation, modification, and movement, helping identify potential threats such as data exfiltration, insider misuse, and file tampering.

---

2. Problem Statement

Organizations face multiple risks related to file handling:

- Unauthorized copying of sensitive data
- Insider threats and data leakage
- Malware modifying or replacing files
- Untracked file transfers to external storage

Without proper monitoring, these activities can go undetected, leading to serious security incidents.

---

3. Objectives

- Monitor real-time file system activity
- Detect sensitive file access
- Identify high-risk file types

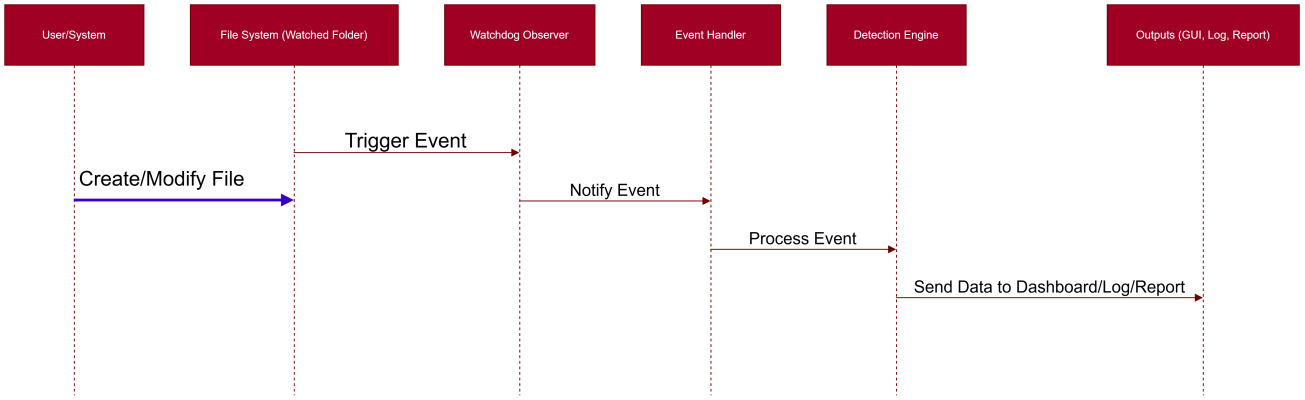
- Perform file integrity checks using hashing
- Generate alerts for suspicious behavior
- Maintain logs and generate audit reports

---

4. System Architecture

The system follows an event-driven architecture where file system events are captured and processed through multiple layers.

Architecture Flow



---

5. Tools and Technologies

Programming Language

- Python

Libraries Used

- watchdog – file system monitoring
- hashlib – SHA-256 hashing
- tkinter – graphical user interface

Platform

- Windows Operating System

---

6. Features

6.1 Real-Time Monitoring

Tracks file system events such as:

- File creation
- File modification
- File movement

6.2 Sensitive File Detection

The system identifies sensitive files based on extensions:

- `.pdf`, `.docx`, `.xlsx`, `.zip`, `.pem`, `.key`

6.3 File Integrity Verification

Each file is processed using SHA-256 hashing to detect:

- Unauthorized modifications
- File tampering

6.4 High-Risk File Detection

High-risk files such as:

- `.zip` (compressed data)
- `.pem`, `.key` (credential files)

are flagged as potential threats.

6.5 Bulk Activity Detection

If multiple file events occur within a short period, the system flags it as suspicious activity.

6.6 Logging System

Each event is logged with:

- Timestamp
- File name
- Event type
- File size
- Hash value
- Alert status

6.7 Report Generation

The system generates a structured report containing:

- Summary of events
- Number of sensitive files
- Alerts triggered
- Detailed event logs

---

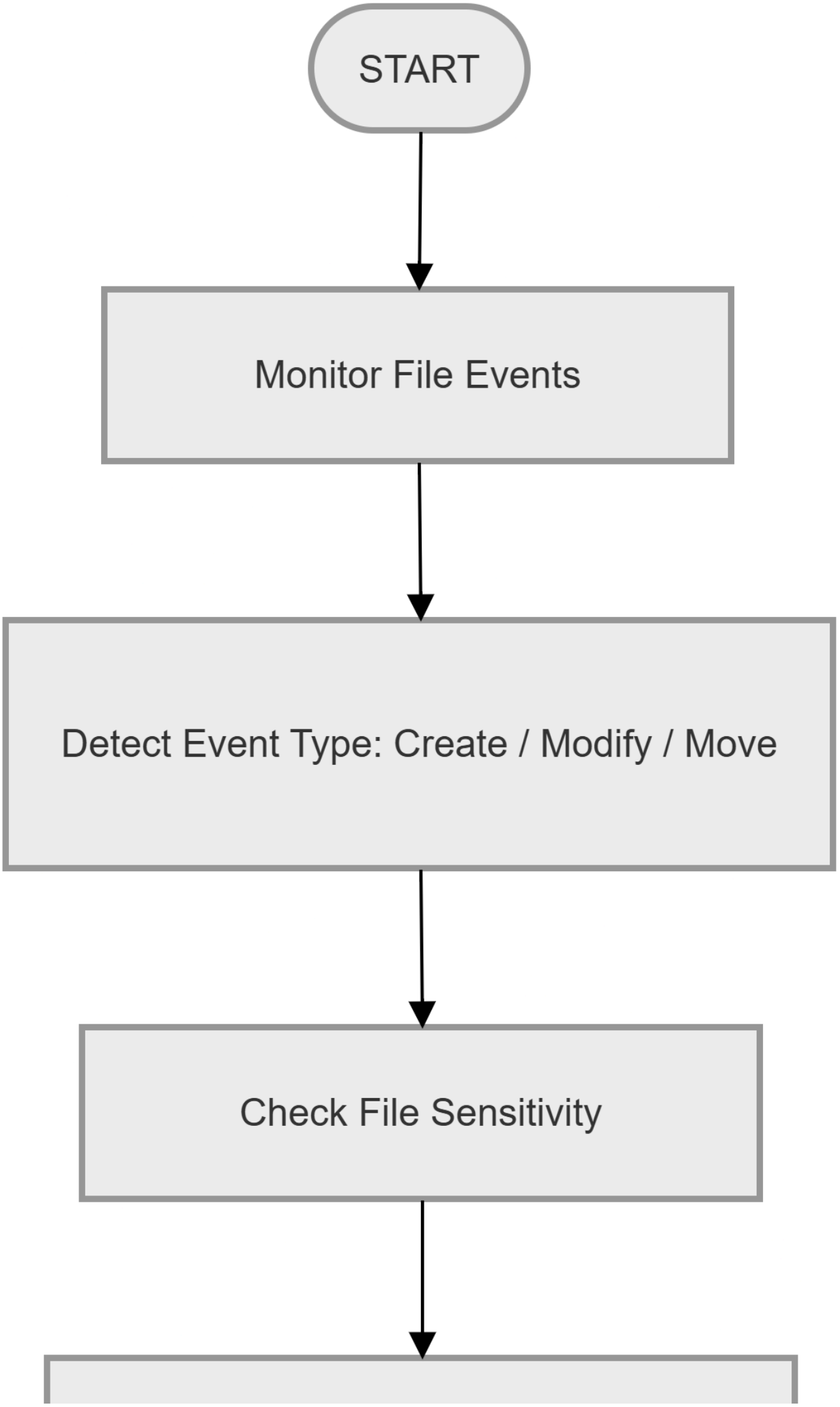
7. Implementation Details

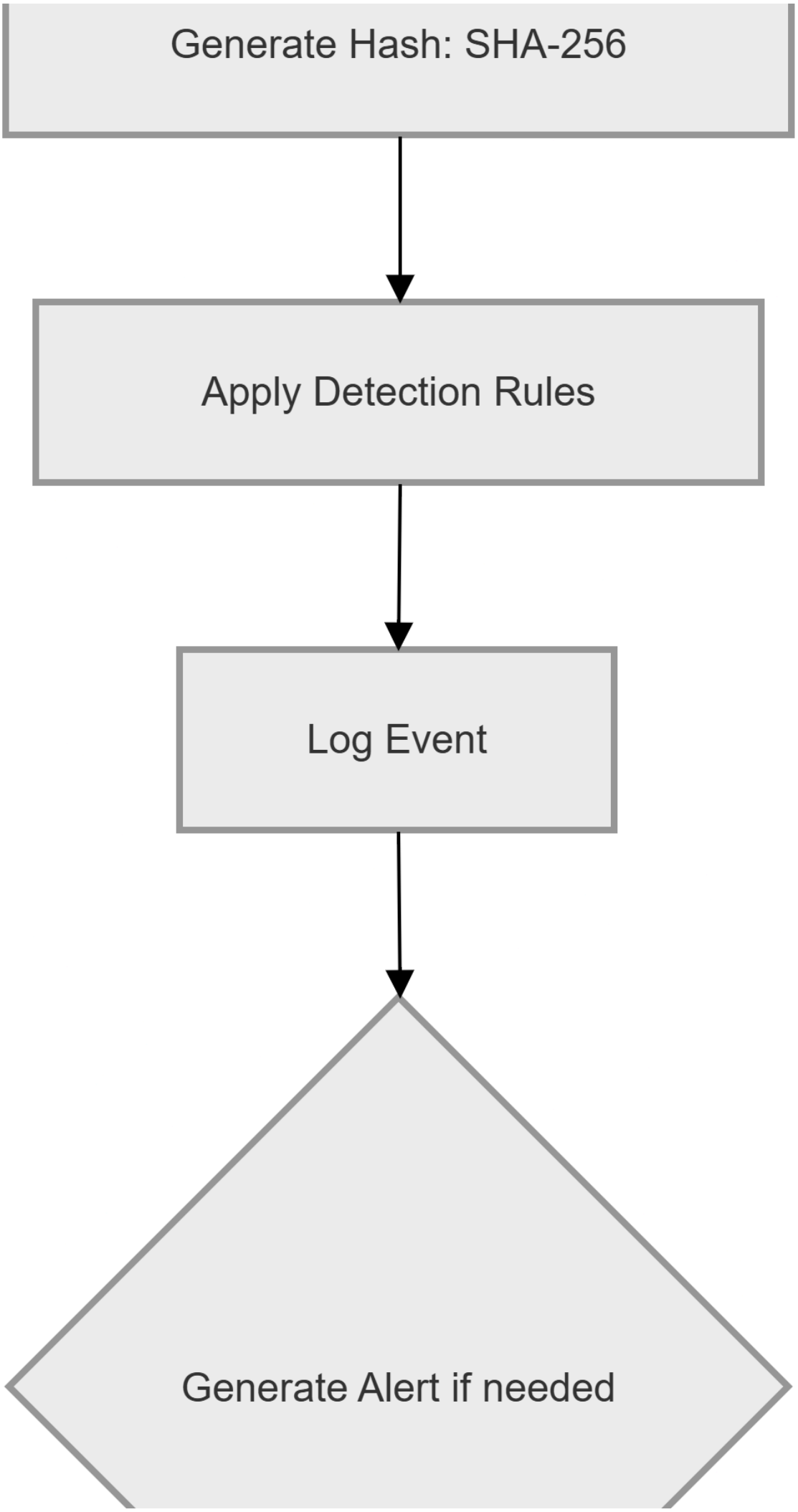
The system is implemented using a modular approach:

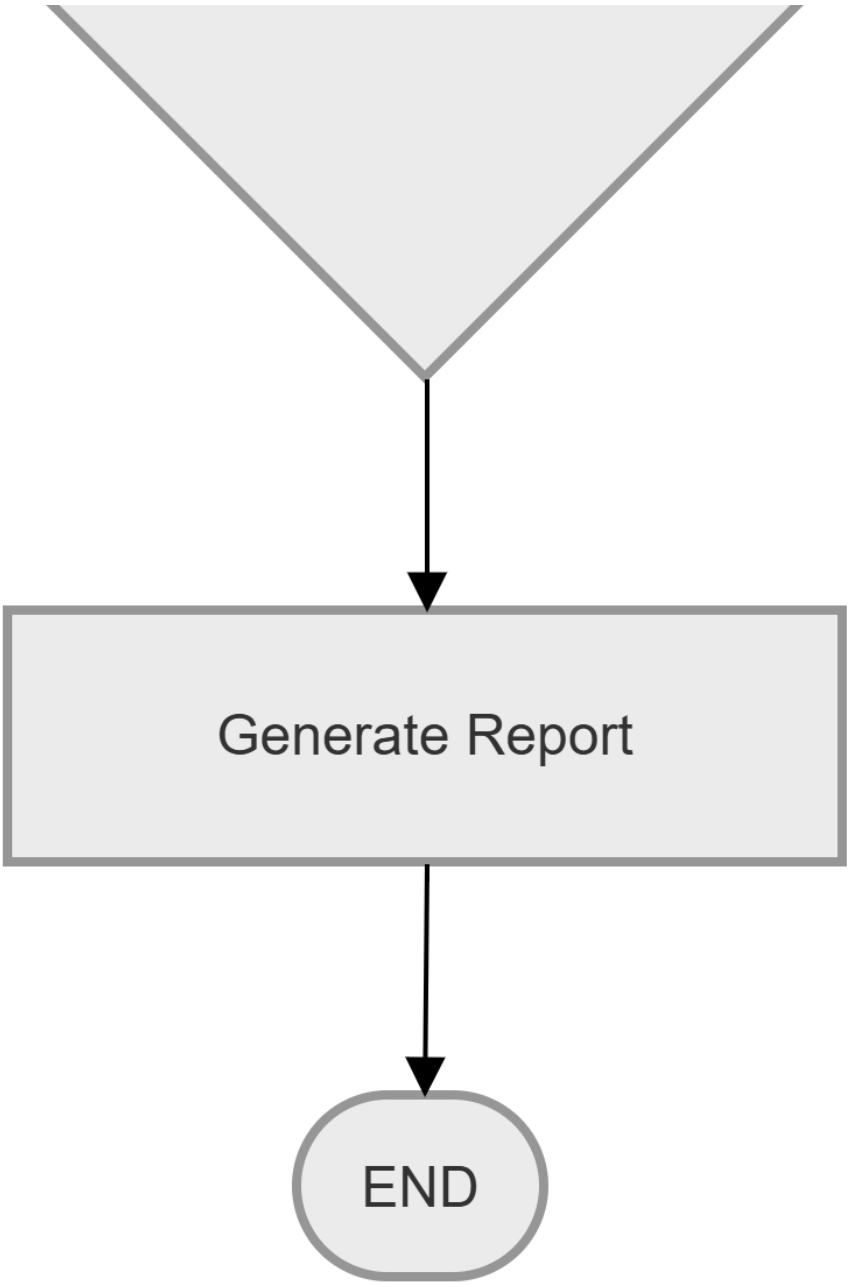
- A GUI dashboard is built using tkinter
- Watchdog observer monitors file system activity
- Event handlers process file events
- Detection logic identifies suspicious behavior
- Hashing ensures file integrity
- Events are stored and used for reporting

---

8. Workflow


START
Monitor File Events
Detect Event Type: Create / Modify / Move
Check File Sensitivity


Generate Hash: SHA-256
Apply Detection Rules
Log Event
Generate Alert if needed



---

9. Code Highlights

9.1 Hashing Function

```
def calculate_hash(file_path):
    with open(file_path, "rb") as f:
        return hashlib.sha256(f.read()).hexdigest()
```

9.2 Event Handling Logic

```
def on_created(self, event):
    if not event.is_directory:
        self.process(event.src_path, "CREATED")
```

9.3 Detection Logic

```
if filename.endswith(('.zip', '.pem', '.key')):
    status = "HIGH RISK"
elif filename.endswith(('.pdf', '.docx')):
    status = "ALERT"
```

10. Output

```
The system generates:
```

- Real-time event logs
- Alerts for suspicious file activity
- File integrity verification results
- Final audit report (text file)

11. Sample Log Output

```
SECURE FILE MONITOR REPORT
========================================

Generated: 2026-03-17 12:01:04.739640

SUMMARY
Total Events: 5
Sensitive Files: 5
Alerts: 5

EVENT DETAILS
----------------------------------------
2026-03-17T12:00:56.944460
CREATED | XSS Payload List - Copy-Ready Plain Text Format.docx | 439.5KB
Hash: ERROR
Sensitive: True | Alert: True
----------------------------------------
2026-03-17T12:00:56.945460
MODIFIED | XSS Payload List - Copy-Ready Plain Text Format.docx | 439.5KB
Hash: ERROR
Sensitive: True | Alert: True
----------------------------------------
2026-03-17T12:00:56.946966
MODIFIED | XSS Payload List - Copy-Ready Plain Text Format.docx | 439.5KB
Hash: ERROR
Sensitive: True | Alert: True
----------------------------------------
2026-03-17T12:00:56.946966
MODIFIED | XSS Payload List - Copy-Ready Plain Text Format.docx | 439.5KB
Hash: ERROR
Sensitive: True | Alert: True
----------------------------------------
2026-03-17T12:00:56.947972
MODIFIED | XSS Payload List - Copy-Ready Plain Text Format.docx | 439.5KB
Hash: ERROR
Sensitive: True | Alert: True
----------------------------------------
```

12. Limitations

```
Does not monitor network-level file transfers
```

- Cannot identify the user performing the action
- Limited to selected directories only

13. Future Enhancements

- USB device detection
- Email alert system
- CSV/JSON log export
- SIEM integration (e.g., Wazuh)
- Advanced anomaly detection

- USB device detection

- Email alert system
- CSV/JSON log export
- SIEM integration (e.g., Wazuh)
- Advanced anomaly detection